Plano do App Scanner de PDF (Android)

Oct 14, 2026 · @Vinicius

Visão geral e decisões

O app será nativo Android em Kotlin, 100% offline e gratuito: abre a câmera, recorta o documento automaticamente, gera o PDF e permite compartilhar ou salvar. O núcleo do escaneamento usará o **ML Kit Document Scanner** do Google, a mesma tecnologia usada no Google Drive e na câmera dos Pixel (documentação).

Tema	Decisão	Motivo
Plataforma	Somente Android, distribuído pela Google Play	Escopo definido pelo projeto
Linguagem	Kotlin	Linguagem oficial do Android, melhor suporte
Interface	Jetpack Compose + Material 3	Padrão atual do Google para UI nativa
Back-end	Nenhum servidor; tudo roda no aparelho	Custo zero, mais privacidade, funciona sem internet
Escaneamento	ML Kit Document Scanner	Câmera, recorte automático e correção de perspectiva prontos
Monetização	Gratuito, sem anúncios	Decisão do projeto para a v1
Escopo da v1	Foto → recorte automático → PDF → compartilhar/salvar	Produto mínimo, publicável rápido

Fora da v1 (planejado para depois): várias páginas por PDF, filtros de imagem e OCR.

Stack tecnológica

Como não há servidor, o "back-end" deste app é a **camada de dados local**: banco de dados no aparelho, arquivos PDF no armazenamento interno e o processamento de imagem feito pelo Google Play services.

Front-end (o que o usuário vê)

Peça	Tecnologia	Para que serve
Linguagem	Kotlin 2.x	Todo o código do app
Interface	Jetpack Compose + Material 3	Telas, botões, listas, tema claro/escuro
Navegação	Navigation Compose	Troca entre telas
Estado da tela	ViewModel + StateFlow (Coroutines)	Mantém os dados da tela e reage a mudanças
Miniaturas	Coil + PdfRenderer (nativo do Android)	Mostra a primeira página de cada PDF na lista
Splash	SplashScreen API (androidx.core)	Tela de abertura padrão do Android

"Back-end" local (o que roda por trás, no aparelho)

Peça	Tecnologia	Para que serve
Escaneamento	ML Kit Document Scanner (play-services-mlkit-document-scanner)	Abre a câmera, detecta bordas, recorta e já entrega o PDF
Banco de dados	Room (SQLite)	Guarda nome, data, tamanho e caminho de cada documento
Preferências	DataStore	Tema, padrão de nome de arquivo
Arquivos	Armazenamento interno do app (filesDir)	Onde os PDFs ficam guardados com segurança
Compartilhamento	FileProvider + Intent de compartilhamento	Envia o PDF para WhatsApp, Telegram, e-mail etc.
Salvar fora do app	Storage Access Framework (CreateDocument)	Usuário escolhe onde salvar (Downloads, Drive...)
Injeção de dependências	Hilt	Organiza as dependências entre as camadas

Ferramentas e configuração

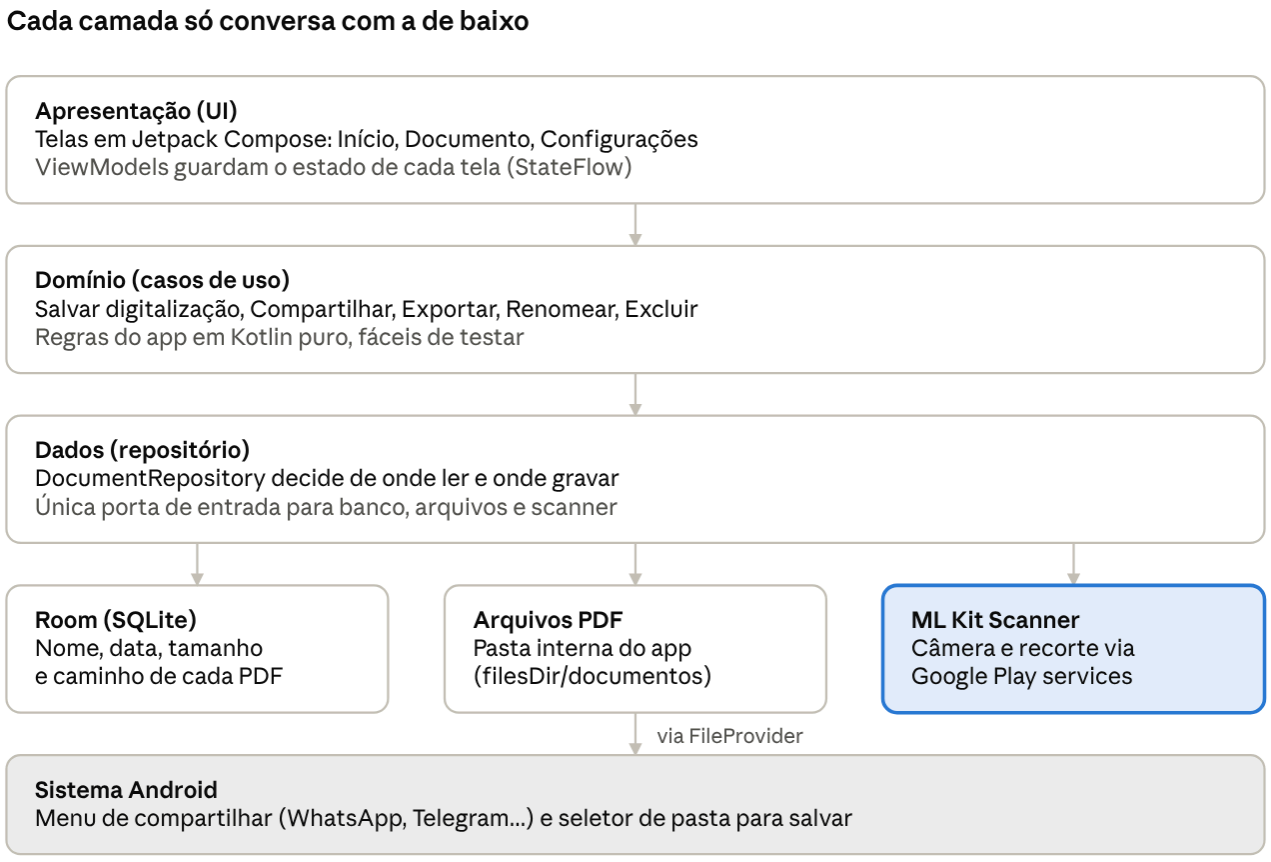
- **IDE:** Android Studio (versão estável mais recente)

- **Build:** Gradle com Kotlin DSL + catálogo de versões (libs.versions.toml)
- **targetSdk / compileSdk:** 36 (Android 16), exigido pela Google Play para apps novos desde 31/08/2026 (requisito oficial)
- **minSdk:** 24 (Android 7.0), cobre praticamente todos os aparelhos ativos; o ML Kit exige no mínimo 21
- **Testes:** JUnit + MockK (lógica), Compose UI Test (telas)
- **Monitoramento de falhas:** Android Vitals do Play Console, sem SDK extra e sem enviar dados a terceiros

Uma vantagem do ML Kit: o fluxo de câmera roda dentro do Google Play services, então o app não precisa pedir permissão de câmera nem de armazenamento. Isso simplifica a política de privacidade e o formulário de Segurança dos Dados.

Arquitetura

A arquitetura é MVVM com Clean Architecture simplificada, num único módulo: suficiente para um app pequeno, mas organizada para crescer (OCR, filtros, várias páginas) sem reescrever nada.



arquitetura · 3 camadas + fontes de dados locais

A interface nunca acessa banco, arquivos ou scanner diretamente; tudo passa pelo repositório. O ML Kit (em destaque) é a única parte que depende do Google Play services.

Estrutura de pastas

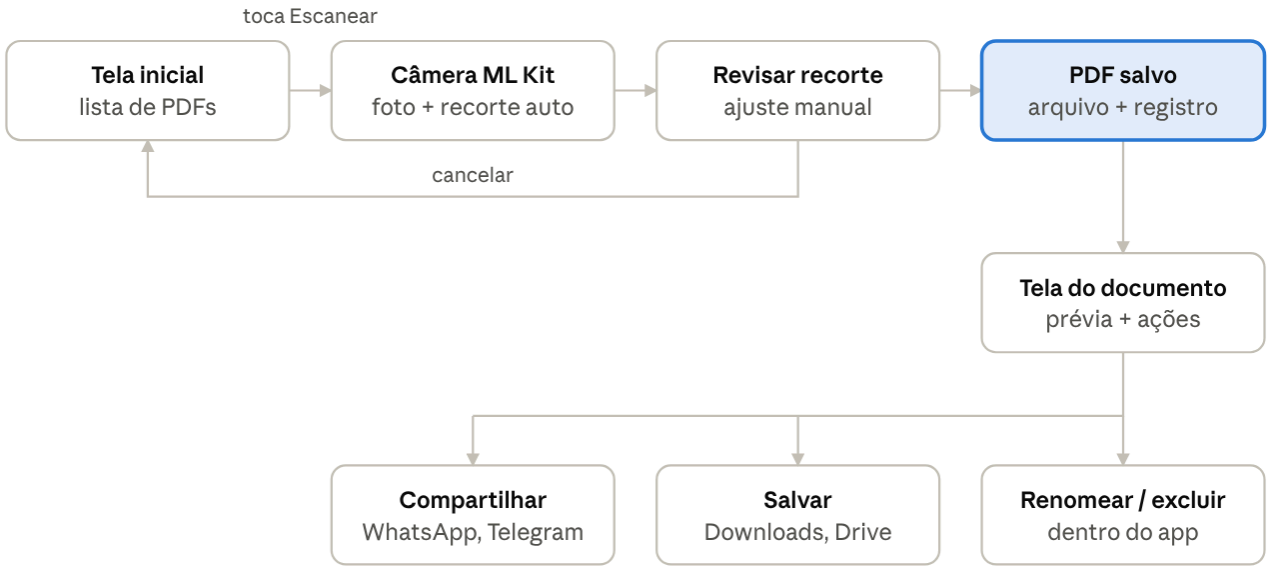
```
app/src/main/java/br/com/seuapp/scanner/
├── ScannerApp.kt          Application + Hilt
├── MainActivity.kt        ponto de entrada, tema e navegação
├── ui/
│   ├── theme/             cores, tipografia, tema claro/escuro
│   ├── navigation/        rotas entre telas
│   ├── home/              HomeScreen + HomeViewModel
│   ├── document/          DocumentScreen + DocumentViewModel
│   └── settings/          SettingsScreen + SettingsViewModel
├── domain/
│   ├── model/             Document
│   └── usecase/           SaveScan, ShareDocument, ExportDocument,
RenameDocument, DeleteDocument
├── data/
│   ├── local/             Room: DocumentEntity, DocumentDao, AppDatabase
│   ├── files/             PdfFileStorage: copiar, renomear, apagar PDFs
│   ├── prefs/             DataStore: tema, padrão de nome
│   └── repository/        DocumentRepositoryImpl
├── scanner/               DocumentScannerLauncher: configura e abre o ML Kit
└── di/                    módulos Hilt
```

Configuração do scanner na v1: modo base (SCANNER_MODE_BASE), limite de 1 página, importação da galeria desativada e saída apenas em PDF, para gerar o arquivo mais rápido.

Fluxo do usuário e telas

O app tem apenas três telas próprias; a câmera e o recorte são fornecidos pelo ML Kit.

Do toque em Escanear ao PDF compartilhado num único fluxo



fluxo do usuário · da abertura ao compartilhamento

Se o usuário cancelar na câmera ou no recorte, volta para a tela inicial sem criar nada. Ao confirmar, o PDF é salvo e a tela do documento abre direto, com o botão Compartilhar em destaque.

Tela	O que mostra	Ações
Tela inicial	Lista dos PDFs (miniatura, nome, data, tamanho), mais recentes primeiro; estado vazio na primeira vez	Escanear, abrir documento, menu rápido de compartilhar/renomear/excluir
Câmera e recorte (ML Kit)	Visor com detecção de bordas, depois ajuste dos cantos	Fotografar, ajustar, confirmar ou cancelar
Documento	Prévia da página, nome editável, tamanho do arquivo	Compartilhar, Salvar no aparelho, Abrir em outro app, Excluir
Configurações	Tema, padrão de nome dos arquivos, política de privacidade, versão	Alterar preferências

Design

O design segue o Material 3 (Material You) com um único objetivo: chegar ao PDF em no máximo dois toques a partir da abertura do app.

- **Ação principal sempre visível:** um botão grande "Escanear" (Extended FAB) na parte inferior da tela inicial, ao alcance do polegar.
- **Cores dinâmicas:** no Android 12 ou superior, o app adota as cores do papel de parede do usuário; em versões antigas, usa uma paleta própria (sugestão: azul-petróleo como cor primária, que remete a documento e confiança).
- **Tema claro e escuro** automáticos, seguindo o sistema, com opção manual nas Configurações.
- **Tipografia:** a fonte padrão do sistema (Roboto), sem fontes extras, para manter o app leve.
- **Estado vazio amigável:** na primeira abertura, ilustração simples + frase "Toque em Escanear para criar seu primeiro PDF".
- **Acessibilidade:** áreas de toque de no mínimo 48 dp, descrições para leitores de tela (TalkBack) e contraste adequado.
- **Idioma:** português do Brasil na v1, com todos os textos em strings.xml para facilitar traduções futuras.
- **Ícone:** ícone adaptativo (adaptive icon) com uma folha e um canto dobrado ou moldura de recorte; precisa ser original, sem lembrar a marca de concorrentes.

As telas de câmera e de ajuste do recorte são desenhadas pelo próprio Google (ML Kit) e não podem ser personalizadas; o restante do app é 100% do nosso design.

Compartilhar e salvar

Não é preciso integrar WhatsApp, Telegram ou qualquer rede individualmente: o menu de compartilhamento do Android já lista automaticamente todos os apps instalados que aceitam PDF.

Ação	Como funciona	Permissão necessária
Compartilhar	FileProvider gera um link seguro e temporário para o PDF; Intent ACTION_SEND com tipo application/pdf abre o menu do sistema (WhatsApp, Telegram, Gmail, Drive, Bluetooth...)	Nenhuma

Ação	Como funciona	Permissão necessária
Salvar / Baixar	Storage Access Framework abre o seletor de pasta do Android; o usuário escolhe Downloads, Drive ou outra pasta	Nenhuma
Abrir em outro app	Intent ACTION_VIEW abre o PDF no leitor padrão do celular	Nenhuma
Renomear / excluir	Feito dentro do app, atualizando o arquivo e o registro no Room	Nenhuma

Detalhes de implementação:

1. O PDF é gerado pelo ML Kit e copiado para a pasta interna do app (filesDir/documentos) com um nome padrão, por exemplo "Documento_2026-10-14_2301.pdf".
2. Um registro é criado no Room com nome, data, tamanho e caminho do arquivo.
3. Ao compartilhar, o app usa Intent.createChooser para mostrar sempre o menu completo, com o nome do arquivo visível no destino.
4. Ao salvar, o app copia o arquivo para o local escolhido pelo usuário; o original continua no histórico do app.

Publicação na Google Play

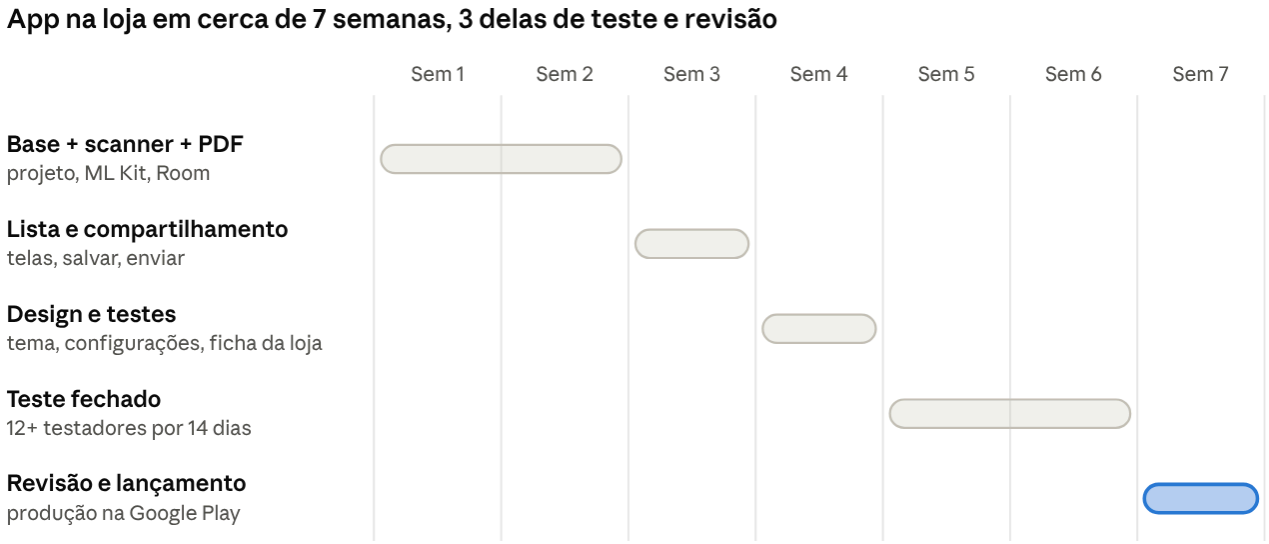
O maior gargalo da publicação não é técnico: contas pessoais novas precisam de um teste fechado com pelo menos 12 testadores inscritos por 14 dias seguidos antes de liberar a produção (regra oficial). Contas de organização (empresa) não têm essa exigência.

- [ ] Criar conta de desenvolvedor no Play Console (taxa única de US$ 25) e verificar identidade
- [ ] Escolher um nome original para o app, sem usar ou imitar "CamScanner" (marca registrada)
- [ ] Gerar o pacote em formato AAB (Android App Bundle) com Play App Signing ativado
- [ ] Configurar targetSdk 36
- [ ] Publicar uma política de privacidade simples (pode ser uma página gratuita) declarando que nenhum dado é coletado
- [ ] Preencher o formulário de Segurança dos Dados: nenhum dado coletado ou compartilhado
- [ ] Responder ao questionário de classificação indicativa e definir o público-alvo

- [ ] Preparar a ficha da loja: ícone 512×512, imagem de destaque 1024×500, pelo menos 2 capturas de tela, descrição curta e longa
- [ ] Rodar teste interno (equipe, sem revisão) para iterar rápido
- [ ] Rodar teste fechado com 15 a 20 testadores (margem acima dos 12, porque se alguém sair a contagem de 14 dias recomeça)
- [ ] Solicitar acesso à produção e aguardar a revisão do Google

Cronograma, riscos e próximas versões

Com uma pessoa desenvolvendo em tempo integral, a v1 leva cerca de 4 semanas de código e mais 3 semanas de teste fechado e revisão: aproximadamente 7 semanas até estar na loja (estimativa).



cronograma estimado · 5 fases em 7 semanas

As fases 1 a 3 são código; as fases 4 e 5 dependem da Google Play e não podem ser aceleradas, então vale recrutar os testadores ainda durante a fase 3.

Riscos e como lidar

Risco	Impacto	Como lidar
Primeiro uso exige baixar o módulo do scanner pelo Play services	Usuário sem internet na primeira abertura não consegue escanear	Verificar disponibilidade com ModuleInstallClient e mostrar mensagem clara pedindo conexão uma única vez

Risco	Impacto	Como lidar
Telas de câmera e recorte não são personalizáveis	Visual da câmera é o padrão do Google	Aceitável na v1; se for essencial ter câmera própria, migrar para CameraX + OpenCV (cerca de +3 semanas)
Teste fechado não atingir 12 testadores por 14 dias	Publicação atrasa	Recrutar 15 a 20 pessoas antes de começar o teste
PDFs grandes demais para o WhatsApp	Envio lento	Ajustar qualidade JPEG no ML Kit e mostrar o tamanho do arquivo antes de compartilhar
Atualização anual do targetSdk pela Google Play	App some para novos usuários se ficar desatualizado	Reservar uma revisão por ano (normalmente até agosto)

Próximas versões

- **v1.1:** várias páginas por PDF (o ML Kit já suporta; basta liberar o limite de páginas)
- **v1.2:** filtros de imagem (modo preto e branco, realce) usando o modo completo do ML Kit
- **v1.3:** importar foto da galeria
- **v2.0:** OCR com ML Kit Text Recognition (offline), PDF pesquisável, juntar e dividir PDFs, proteção por senha